

A2: Image Filtering and Edge Detection Analysis

This assignment explores image filtering and edge detection methods using Python and OpenCV. The image 'lena' was processed through both custom implementations and OpenCV functions, providing practical experience in image processing.

Comparison of Custom and OpenCV Implementations

Starting with the averaging filter, both the custom implementation and OpenCV's `cv2.filter2D` gave similar smoothing effects, which reduced noise and softened the image. However, OpenCV's output displayed a more uniform blur compared to the implementation from scratch. This is because of OpenCV's better convolution that uses more precise measures for the averaging filter. My custom method, which included manual calculations gave slight inconsistencies with pixels and the blur is just a bit less smooth. (See Appendix A.1)

The Gaussian filter also showed minimal visual differences between my custom code and OpenCV's `cv2.GaussianBlur`. Both gave a slight blur to the original image, smoothing it. OpenCV's output looked more evenly applied, and was noticeable especially around the edges, due to precomputed Gaussian kernels. My custom implementation calculated the Gaussian filter manually, which led my image to look slightly more smooth and less consistent overall. (See Appendix A.2)

The Sobel sharpening filter results highlighted the biggest differences overall. In OpenCV's output, edges were sharply defined, and the background was free of most noise, enhancing the edges' contrast. Similarly, the horizontal and vertical filters were smooth and free of noise and the edges, especially around the hat- were more pronounced. My custom Sobel filter, however, was noisier, with a grainy texture in areas that should be smooth, like the background. The edges were less distinct, blending more into the noisy background. This likely comes from my custom implementation lacking OpenCV's optimized gradient calculation, which reduces noise and sharpens edges through non-maximum suppression. (See Appendix A.3)

The main reason for these differences is OpenCV's efficient algorithms, which suppress weaker gradients, reduce noise, and enhance edge clarity. My custom code didn't have these optimizations, leading to a noisier and less defined output, especially seen in the Sobel sharpening filter.

Comparison of the Marr-Hildreth and Canny Edge Detectors

The Marr-Hildreth and Canny edge detectors produced quite different results. The Marr-Hildreth method outputted a thicker, less precise edge map due to its Laplacian-based approach. This method identifies bigger regions but also loses precision, making edges appear more connected and less localized (See Appendix C.1). On the other hand, the Canny detector generated a cleaner, sharper edge map. This is because of the multi-step process it uses: gradient calculation, non-maximum suppression, and double thresholding, which yields sharper, more defined edges and is better suited for applications needing more precise edge detection. (See Appendix C.2)

The main driver behind the two's differences is their algorithms. Canny's steps focus on refining and isolating edges, Marr-Hildreth's simpler gradient calculation leads to broader, less defined edges. Canny's method is better at capturing fine details, making it more effective for detailed feature extraction, and should be used by default.

Connected Components in the Edge Maps (Task 4)

In Task 4, the Group Adjacent Pixels algorithm from the lecture was used to identify connected components in the edge maps in the Marr-Hildreth and Canny detectors. The Marr-Hildreth edge map produces thicker edges, which are more fragmented and lack continuity. The Canny edge detector on the other hand, was more connected, smooth and formed well defined lines. (See Appendix D)

The Canny edge map gives better feature representation, since it has clean, well-defined boundaries. It minimizes non-edge pixels and focuses on specific features, which allows for more sharp lines. While the Marr-Hildreth map may work for broader feature representation, Canny is preferable when feature accuracy is needed.

Performance and Efficiency (Bonus)

The efficiency comparison between the from-scratch implementations and OpenCV's built-in functions demonstrate that my custom implementations were significantly slower, highlighting the performance advantages of OpenCV's optimized functions.

For the Averaging Filter, the custom implementation took approximately 1.015751 seconds, while the OpenCV version executed almost instantly in 0.000000 seconds, resulting in a time difference of 1.015751 seconds. This difference shows the efficiency of OpenCV for this operation. Similarly, the Gaussian Filter showed a custom processing time of 1.965741 seconds, compared to 1.965741 seconds for OpenCV, leading to a time difference of 1.964809 seconds.. Lastly, the Sobel Filter had a processing time of 3.432578 seconds for the code from scratch, while OpenCV completed the same task in just 0.004987 seconds, a time difference of 3.427591 seconds. These differences show the effectiveness of OpenCV in providing fast responses, which is crucial for real-time image processing applications.

Overall, the comparisons show that while custom implementations may offer flexibility and ways to learn, OpenCV's implementations are much faster and efficient, which is why they are preferred for practical applications. (See Appendix E)

Conclusion

In conclusion, while the custom implementations were able to produce comparable results to OpenCV in terms of visual output, the significant differences in performance highlight the benefits of using OpenCV. OpenCV's optimized functions are much faster, making it a preferred choice for real-time or near real-time image processing requirements.

Appendices

Appendix A: Operations from Scratch

1. Averaging Smoothing Filter

Original Image



Averaged Image



2. Gaussian Smoothing Filter

Original Image



Gaussian Smoothed Image



3. Sobel Sharpening Filter

Original Image



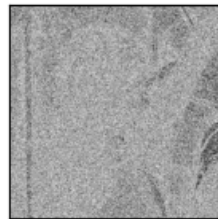
Sobel Horizontal



Sobel Vertical



Sobel Edge Detection



Sobel Sharpened Image



Appendix B: Operations with OpenCV

1. Averaging Smoothing Filter

Original Image



Averaged Image (cv2)



2. Gaussian Smoothing Filter

Original Image

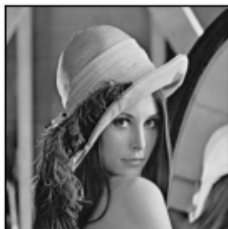


Gaussian Smoothed Image (cv2)

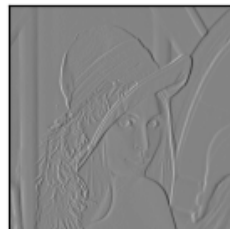


3. Sobel Sharpening Filter

Original Image



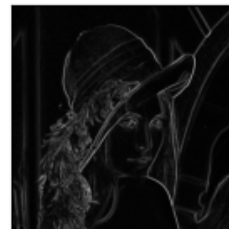
Sobel Horizontal cv2



Sobel Vertical cv2



Sobel Edge Detection cv2



Sobel Sharpened Image cv2

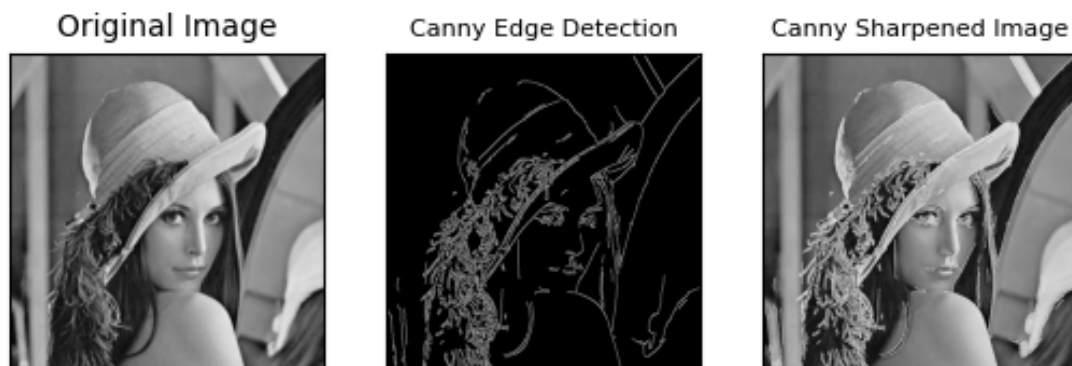


Appendix C: Marr-Hildreth and Canny Edge Detectors

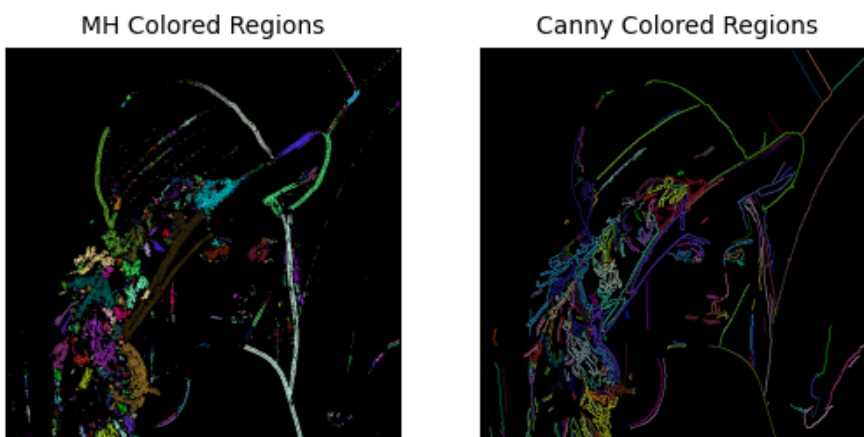
1. Marr-Hildreth Edge Detection



2. Canny Edge Detection



Appendix D: Edge Maps and Connected Regions



Appendix E: Efficiency Table

Task	From-Scratch Time (s)	OpenCV Time (s)	Time Difference (s)
Averaging Filter	1.287585	0.000000	1.287585
Gaussian Filter	1.876680	0.000000	1.876680
Sobel Filter	3.751530	0.005982	3.745548